

### **Q1. What are the major screenreaders there are to test?**

A: There are a few major screenreaders:

- 1) MAC OS - Voiceover (best with Safari)
- 2) Windows - JAWS, Narrator, NVDA,
- 3) LINUX - ORCA
- 4) Chromevox - Chrome Browser
- 5) Android - TalkBack

### **Q2. We want to hide content from the screen readers how you will do?**

A: There are a few ways through which we can hide the content from the screen readers:

- 1) Hiding images: `aria-hidden="true"` and `role="presentation"`
- 2) To completely hide from the screen readers use `display:none`

### **Q3. Is infinite scrolling accessibility friendly?**

A: No, it is not accessible at all. We should have the data loading only on-demand such as on click of the button

### **Q4. What are the benefits of having regions or landmarks in your code?**

A: VoiceOver users can navigate a page via Aria landmarks by choosing Landmarks in the Rotor.

Such as headings, header, footer, main, etc.

For Mac here is the shortcut key to activate the Rotor:

`Control + Alt + U`

To Navigate the list of Landmarks:

`Left/Right Arrow`

To Navigate through the list of Landmarks:

`Up/Down Arrow`

## role="presentation" vs aria-hidden

If you are accessibility enthusiastic then you must be aware of the `role = presentation` and `aria-hidden`. There is always confusion that what is the difference between them? As at the high level, both are used to hide the content from the assistive technologies. Then why we need both? For this blog, we need a voice over. I am using Mac OS, Safari browser, and default Mac voice over. And accessibility DOM, I am using Mozilla, and Chrome browser on Mac OS.

PS: If you are using the Mac system please prefer to use the safari browser over chrome or Firefox for Voice Over (screenreaders).

### What is role="presentation"

For accessibility, it is important to write semantic code. However, there are many use-cases where we don't have any semantic tag available in the HTML in such a case. We use the `role` attribute to make those identified by the assistive technologies and browsers.

```
1  <!Doctype html>
2  <head>
3  <title>aria-hidden vs presentation </title>
4  </head>
5  <body>
6  <h1>ally</h1>
7  <h1 role="presentation">Welcome</h1>
8  <button role="presentation">read more...</button><br />
9  
10 <h2 role="presentation">hello</h2>
11 <a role="presentation">Google</a>
12 <ul role="presentation">
13   <li>apple</li>
14   <li>Orange</li>
15 </ul>
16 </body>
17 </html>
```

Now, there are cases where we want to hide the few elements from the assistive technologies. Such as icons, images, or any elements. In such cases, we use `role=presentation`.

Adding attribute `role="presentation"` will hide it from the assistive technologies but the main difference comes is what `role="presentation"` does under the hood?

`role="presentation"` will remove all the semantic information from the element from Accessibility DOM. What does it mean? It means that screen-readers will treat every element as "Text element" rather than 'heading, image, etc..'. but if the elements are focusable the semantic won't be

removed at all. It means for `button, a, input` `role="presentation"` won't have any effect.

| Role          | Name           | Checks                   |
|---------------|----------------|--------------------------|
| ▼ document:   | "Test"         | No checks for this node. |
| text leaf:    | "Welcome"      | ▼ Properties             |
| ► pushbutton: | "read more..." | name: "Test"             |
| whitespace:   | "\n"           | role: "document"         |
| text leaf:    | "hello"        | ▼ actions: []            |
| text leaf:    | "Google"       | length: 0                |
|               |                | value: ""                |
|               |                | DOMNode: #document ⚙     |
|               |                | description: ""          |
|               |                | keyboardShortcut: ""     |
|               |                | childCount: 5            |
|               |                | indexInParent: 1         |
|               |                | ► states: [...]          |
|               |                | ► relations: [...]       |

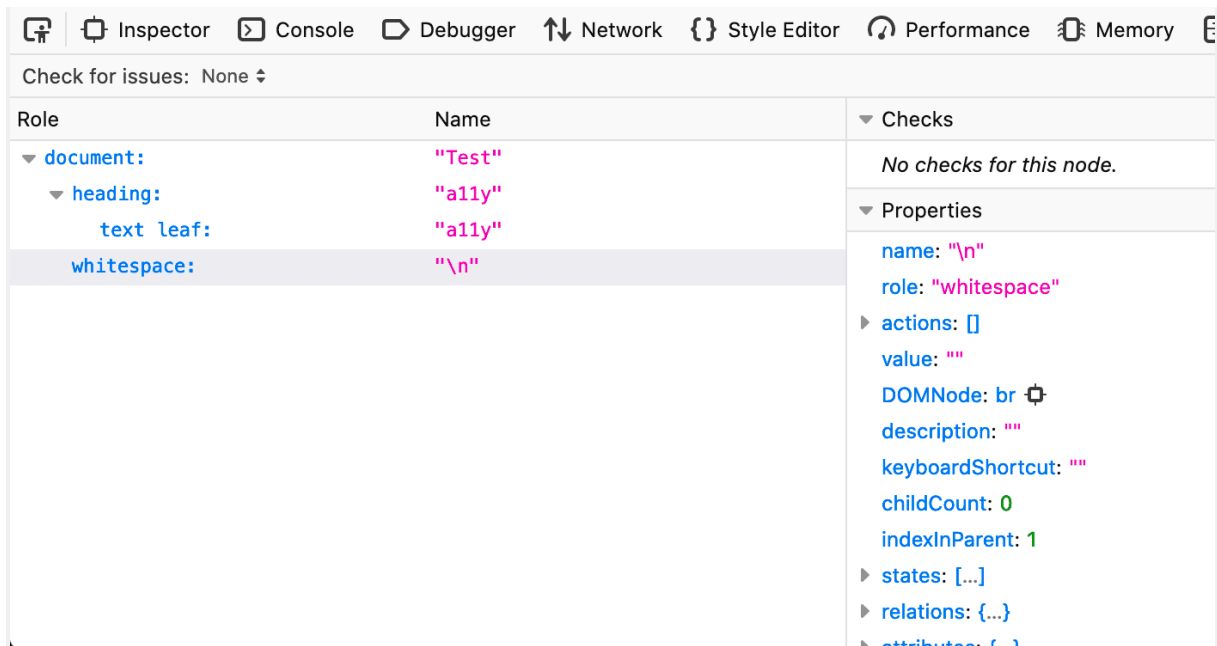
## What is aria-hidden="true"

ARIA (Accessible Rich Internet Application) another way to have accessible elements. If you don't know what ARIA is then read my blog on it here.

```
1 <!doctype html>
2 <head>
3 <title>aria-hidden vs presentation </title>
4 </head>
5 <body>
6 <h1>a11y</h1>
7 <h1 aria-hidden="true">Welcome</h1>
8 <button aria-hidden="true">read more...</button><br />
9 
10 <h2 aria-hidden="true">hello</h2>
11 <a aria-hidden="true">Google</a>
12 <ol aria-hidden="true">
13 <li>apple</li>
14 <li>Orange</li>
15 </ol>
16 </body>
17 </html>
```

`aria-hidden="true/false"` , is also used to hide the elements from the assistive technologies but this is stricter than the `role="presentation"`. This will remove the element from the Accessibility DOM.

This is used mainly when we are handling the hide/show of the elements from the JavaScript.



## Difference:

Below is the difference between the both attribute on different elements (focusable and non-focusable) with screen-readers. (Tested on MAC OS , Safari Browser , and Voice Over Screenreaders).

|                 | role="presentation"                  | aria-hidden="true"                 |
|-----------------|--------------------------------------|------------------------------------|
| <i>ADOM</i>     | accessibility DOM get generated      | No accessibility DOM get generated |
| <i>image</i>    | Not-readable                         | Not-readable                       |
| <i>list</i>     | Readable with no semantic            | Readable with semantic             |
| <i>headings</i> | Read but semantic removed            | Readable with semantic             |
| <i>button</i>   | Read and didn't removed the semantic | Read and remove the semantic       |

|             | role="presentation"                                 | aria-hidden="true"     |
|-------------|-----------------------------------------------------|------------------------|
| <i>link</i> | Read and semantic only get added when HREF is added | Readable with semantic |

## Now, which to use when?

If you want to remove the semantic values from the element then use the `role="presentation"`. If you are handling the hide/show by JavaScript then use `aria-hidden`.

**Caution: For focusable elements `role="presentation"` doesn't work.**

## Never remove outline from CSS

Many devs disable the outline in CSS by using `{ outline: 0 }`. In my experience I have seen even the testers used to report the issues - *a ugly blue line is coming over the button*.

The sad part is that lack of awareness on accessibility in the tech industry is leading to the non-accessible web. As a result, we go ahead and remove the outline.

## Why outline is important?

When someone tries to access the website by keyboard outline property guides the user about where is the focus currently. If someone will disable outline then there is no way a user will get to know where the focus currently is. This could be highly frustrating.

Defining focus to navigation elements is an accessibility requirement, as per the [Web content accessibility Guidelines](#)  
**2.4.7 Focus Visible: Any keyboard operable user interface has a mode of operation where the keyboard focus indicator is visible. (Level AA)**

## Solution?

Never ever disable outline in CSS. Always have an outline enable. One can leave the default outline style which will be as per the browser and OS or one can make a customized style of outline too.

Below are the examples from dev.to and samshing magazine for the focus style when navigating the site by keyboard.



## alt guide for accessible images

**Alt tags** are the alternative attribute we pass to the images in HTML. *alt* tags are very important for accessibility and SEO. They are the most simple task to do.

*alt* tags are read by the screen readers as well as it will print of the screen when the image is not available.

If you have images in your application it is important to have *alt* tags for the accessibility. But that doesn't mean that you can put anything as *alt* tag. To get 100% out of *alt* tag you should have effective *alt* tags. There are rules for the *alt* tag and we are going to learn that today.

### Type of images & their alt tags

#### 1. Functional Image (Clickable images and image with clickable text)

The images with the link are categorized as functional images. The alt tag of these images should depend upon the context as well as the function of the images.

#### 2. Decorative Images

The images which are just for decorative purpose are categorized as decorative images. The alt tag of such images needs to be evaluated properly. There would be the images which doesn't require alt tag and there would be the images where alt tag is important. Eg: separator images vs icon for 'go to next page'.

#### 3. Logo

It is common that the logo of the website if the image. The general practice is folks put alt tag - "XYZ logo". However, the logo image is also functional,

it provides the link to homepage too. Hence, the appropriate alt tag could be "XYZ homepage".

#### **4. Background Images**

The rule is that content should not be part of the background image. If you have images that have content make them accessible by putting the text in the HTML too.

#### **Tips for alt tags**

1. Should be as per the context and function of the image
2. Should not be redundant
3. Should not start with 'image of...' or 'graphics of...'
4. Every image should have an alt attribute. If there is no alt text then leave it blank. eg: alt=""
5. Alt text of a functional image (e.g., an image within a link) should describe the function as well as the content.
6. Should be descriptive

### **Color contrast rule for disable state**

Color contrast is very important for accessibility. There are many vision-related disabilities people are suffering from. We as developers and designers should make sure that color contrast is as per the accessibility guidelines. However, if you have disable button, links, or any HTML elements on that color contrast rule will not implement.

### **Your Guide for Basic Accessibility**

1. alt tags of the images
2. Correct structure of the HTML page
3. Keyboard focus
4. Captions and subtitles of the video and audio
5. Alternatives of captions such as what is 1+1
6. Not color based messages and UI
7. Responsive
8. proper labels
9. Avoid read more links and add descriptive content
10. Proper color contrast
11. ARIA tags
12. Dynamic Content alert for screen readers
13. Accessible forms: accessible by keyboard, clear criteria for the password, username etc., descriptive error messages, proper labels of input fields.

## Accessible Form Labels

It is important for accessibility to have the label tag with every input tag but there are use-cases where you have the UX - input without any label.

Such as the search field (just with placeholder). This will fail accessibility. If you ask your UX person to re-design the UX there would be a push back. Then how you are going to fix this issue?

A screenshot of the Twitter search bar, which is a light blue rounded rectangle containing a magnifying glass icon and the text "Search Twitter".

Add ARIA-label for the assistive technologies. ARIA-label is the way to tell the assistive technologies that though there is no label tag but this field has this label tag.

```
<input type="text" placeholder="search..." aria-label="search" />
```

## Accessibility Guide for Internationalization





If you are working on a web project which is going to be used in different countries and languages then you need to understand that accessibility plays a big role in that too.

Apart from semantic code, content, color-contrast, there are a few other things to take care of making sure your 'localized' website is accessible to your users.

## **lang attribute**

The first and most important thing to do is to update lang attribute in your HTML page as per the language you will be using.

```
<html lang="en">
```

## **Dates**

Most of the country has a different date format. Eg, India has DD MM YY whereas USA has MM DD YY. When working on i18n make sure that you are changing the date as per the country's preferred format.

Eg: For USA 03/03/2020 means March 3rd 2020 but for Indian, it is 3rd March 2020 and for China, Korea the date format is 2020/03/28

## **Phone number**

Same as dates, phone number formats also differ from country to country. Make sure that as per the country the format of the phone number is also changing.

Eg: For India we have +91-0XXXXXXXXXX but for USA they follow XXX-XXX-XXXX

## **Time Zone**

Time zones are very important. Make sure all the time-related content is getting converted as per the country's TZ or provide a way to user select the timezone.

## **Currency**

If you are an e-commerce website or site dealing in the money. Do make sure you are converting the amount to the country's currency or a way to change the currency.

## **RTL and LTR**

When moving from RTL to LTR the whole experience gets changed. So, make all the assets are also changing accordingly.

## **Graphics**

If the graphics hold the content then make sure those graphics are getting translated to all languages you will be supporting.

## **alt and ARIA**

Just like Graphics, alt and ARIA tags should also get converted to the respective languages.

## What to take care in CSS

CSS is very powerful. A lot can happen through CSS. This is also risky wrt to accessibility. For i18n make sure that there is no fixed width and height of the elements. Content should not be the part of the CSS and if it then does make sure it is getting converted.

## How to do keyboard testing:

- **TAB** key to move to the next element.
- **SPACE** key to select anything.
- **ENTER** key for action.
- **TAB + SHIFT** key to move to the previous element.

## Accessible links on the Web

Every webpage consists of the 'link'. We have internal and external links. The way of creating links in a semantic way is by using anchor tags. However, using the power of the 'JavaScript' developers can make any tag to emulate the anchor tag kind of behaviour.

It doesn't matter which way the developer will follow to create the links on the page but what matters is the 'accessibility'.

Though it is preferable to use semantic tag anchor as it provides a lot of features by default eg: keyboard support by default, screen readers identify them by default etc.

```

<!-- Correct Way -->

<!-- Internal Links -->
<a href="#">post your feedback</a>

<!-- External Links -->
<a href="#" target="_blank">here are the more details</a>

<span onClick="clickHandler()" role="link" tabIndex="0">click here</span>

<!-- Wrong Way -->

<a href="">read more ... </a>

<a href="#">read more ... </a>

<a href="#">click here</a>

<a href="#">link to detail page</a>

<span onClick="clickHandler()">click here</span>

```

## Assistive Technologies and links

It is important to understand how assistive technologies users are going to interact with the links:

- 1) Keyboard users - Keyboard users use `tabs` to navigate from actionable tags and `enter` key to take any action.
- 2) Screen Readers - Screen readers will read the text between the `a` tag and they will prepend the 'link' before the text. Hence, you will hear - "Link Go to the blog detail page".
- 3) Colorblind - For colorblind colors won't matter. They will just click.

## Types of Links

Following are the types of links we can have on any application (not only limited):

- Internal Link
- External Link
- Graphics Link

## Good Practices:

## 1) Content

It is important to make sure the content of the links is clear for the accessibility. Content means the text as linkable.

Following are the things to take care :

- **Do not use 'link' in the content of the link**

For screen readers, this would be redundant information. If you are using anchor tag or role="link", screen readers will prepend 'link' before the text always.

- **Use unique link text**

If you have a lot of links on the page. Keep link text unique as with screen readers will have a bad experience.

- **Avoid capitalization**

Do not capitalize the link content. This will create the problem with the screen-readers. Many times the screen-readers end up reading every letter aloud.

- **Avoid URLs as link text**

URLs are not readable properly. Hence, putting the URLs as link content the screen readers will end up reading the whole URL. Hence, do not put the URLs as the link content

- **Concise length**

The length of the links is debatable. We can make the whole sentence linkable or 1 character too. The decision is to depend upon the developer or the context for the length.

## 2) External link

- **Open in new window**

Always open external links in the new window.

- **Provide information about external links**

Always provide a visual and non-visual cue about the external links. This will help the users to take the decision they want to visit the link or not. Basically external link will navigate the user to new tab or window. Hence, it is important to give them heads up.

## 3) Non-HTML resources

- **Provide information about non-HTML resources**

If the links contain the non-HTML resource such as PDF, excel, ppt, etc. Do provide a visual cue.

```
<a href="report.pdf">PDF Downloadable Report</a>
```

## 4) Graphic Link

- **Use of icons, emojis , ASCII as links**

Provide the alt tags of the icons, emojis, ASCII if using in the links. Screen readers do not understand what the icons and emojis are.

- **Context**

If the image is a link and heading too. Combine them into one. Make them 1 link. This will help the screen reader users to avoid duplicate information.

## 5) Others

- Try to avoid lot of links on the page
- There should be No blank or broken links
- Combine Adjacent Links into a Single Link

## Build Accessible forms

Forms on every website or app are most crucial. The reason is simple they bring the business. If you are an e-commerce website then people would be using it to place orders, or do payment. If you are an education website then people would be using it to fill the admission forms. etc.

A non-accessible form simply means a loss in the business.

Today we are going to learn what makes the forms non-accessible

## Common Mistakes:

### 1) Using only colors for errors and success

Many times UX designers decide to use the colors to communicate the state of the submission of the form - success or errors. This is not accessible by screen readers as well as to the folks who have color blindness problem. Always provide a clear and descriptive reason why the form is not getting submitted successfully.

## DON'T

Please correct the field marked in red

123@we

hello121df

Please correct the field marked in red

123@we

Neha Sharma

## DO

Please correct the following errors:

- Email ID
- Name

123@we

Please enter the valid email id

hello121df

Please enter the valid name

## 2) No descriptive errors

As stated above, always provide descriptive and clear error states. I have seen the errors like : "Please correct the errors", "Please enter valid username", .etc. Form author needs to clearly explain what do you mean by 'valid', which error to be corrected, and what to be corrected?

**DO NOT HAVE GENERIC ERROR MESSAGE**

**HAVE CLEAR & DESCRIPTIVE ERRORS**

## **DON'T**

Please select a valid username

hello@!2!df

## **DO**

Please select a valid username:

- 9 characters from a-z, 0-9
- no special symbols
- all lowercase

hello121df

### **3) Missing descriptive helper text**

To have accessible forms it is mandatory to provide the descriptive and clear helper text. This will help the users to understand the expected input and they can enter without any mistake. Eg: if the expected password is 9 characters without any special symbol then do mention is clearly before the user enters the input.

#### DO NOT HAVE GENERIC HELPER TEXT

#### HAVE CLEAR & DESCRIPTIVE HELPER TEXT

#### DON'T

Please enter username

hello@!2!df

#### DO

Please enter a username:

- 9 characters from a-z, 0-9
- no special symbols
- all lowercase

hello121df

#### 4) No labels of the form

Another common mistake folks do with forms is they do not keep labels separate from the input fields. This is in fact very common and I would say the very favorite style of the UX folks to have labeled as placeholders or as Material design system has label overlapping input fields.

Both are incorrect, reason, labels are important for the screen readers, folks suffering from cognitive, folks who are new to web and need to understand which field is related to what?

The best practice to always have labels.



**DO NOT HAVE LABELS AS  
PLACEHOLDERS**

**HAVE LABELS SEPRATE FROM INPUT  
FIELD**

 **DON'T**

Enter username

Enter password

 **DO**

**Username**

**Password**

**5) Labels are not related with the input**

### DO NOT HAVE NON-LINKABLE LABELS AND INPUT FIELDS

```
<label>Username</label>  
<input type="text" />
```

### HAVE LABELS SEPRATE FROM INPUT FIELD

```
<label for="username">  
Username </label>  
  
<input type="text" id="username" />
```

### DON'T

Username

### DO

Username

How screen readers read the input fields and relate to the `label`? Well, a small HTML attribute does the magic. Now, imagine when there is no attribute. Well, all the screenreaders user end up hearing is - "Input field..". Hence, it is important to link the labels with the input field using the attribute.

## 6) No legends of radios and checkboxes

Legends are very important for `checkbox` and `radios`. The reason is for screenreaders and keyboard supporters. Legends help the screenreaders users to understand

**DO NOT HAVE RADIOS and CHECKBOX WITHOUT LEGEND**

```
<h2>Select your fav colors</h2>
```

```
<label>Red</label>
```

```
<input type="checkbox" name="red" value="red" />
```

...

**DO HAVE LEGEND TO GROUP RADIO and CHECKBOX**

```
<legend>Select your fav colors</legend>
```

```
<label>Red</label>
```

```
<input type="checkbox" name="red" value="red" />
```

...

 **DON'T**

Select your fav colors

☐

RED

☐

GREEN

 **DO**

Select your fav colors

☐

RED

☐

GREEN

## 7) Button for semantic

**DO NOT USE NON-NATIVE ELEMENTS FOR BUTTON**

```
<div>SUBMIT</div>
```

 **DON'T**



**DO USE NATIVE ELEMENT FOR BUTTON**

```
<button type="submit">  
SUBMIT</button>
```

 **DO**



Another common mistake to avoid is not having a semantic `button` tag for the button. The rule here is the same use semantic HTML tags over custom tags. This will help in avoiding a lot of issues such as - focus, screen readers, etc.

## 8) Helper Text or Tool Tips for screen readers

### DO NOT HAVE NON-LINKABLE HELPER TEXT

```
<span>?</span>  
<p>ENTER 9 CHARACTERS</p>
```

### DO HAVE ACCESSIBLE HELPER TEXT

```
<label for="username">  
Username </label>  
  
<input type="text" id="username"  
aria-describedby="userHelp" />  
  
<span>?</span>  
<p id="userHelp">ENTER 9  
CHARACTERS</p>
```

### DON'T

Username



### DO

Username



The most common way of having helper text is having a tool-tip like UI and experience but again the challenge is folks miss out on how screen readers will get to know that what is in the helper text? Well, there is `aria-describedby` to help.

[Here](#) is the code example

## 9) Keyboard support

I kept this at the last, but this is very important to understand and I want to leave you all with this as a closing note. Forms are made up of

- radio, checkbox, dropdown, number input.

Now, how a keyboard user can interact with these? There is the default behavior and every keyboard user is aware of it. So, do not try to break or change the default behavior.

a) Radio: Spacebar key

b) checkbox: Spacebar key

c) dropdown: Enter key to enter the dropdown, arrow keys to select the values and enter key to select

d) number: unless and until you are using arrow icons to indicate up & down to increase and decrease the value

**Here is the code of the Accessible form**

## ARIA

ARIA stands for 'Accessible Rich Internet Application' and is a set of attributes that define ways to make web content and web applications accessible (especially applications made by JavaScript).

## Why

The thumb rule of accessibility is to use HTML5 native tags. However, in today's rich UI experience we know that there are many modules developers are developing for which there are no HTML5 tags. The challenge is how to make these modules identify by the assistive technologies? *ARIA- comes here to make these accessible. Not just helping in identifying, ARIA- also support adding states and properties of the custom module of tag developers are developing.*

ARIA works by changing and augmenting the standard DOM accessibility tree. It doesn't add any keyboard focus or event-listeners to the element.

This can be done by using - role, properties, or state. ARIA-role, ARIA-properties, ARIA-state

## Example

### ARIA (Accessible Rich Internet Application)

role | state | properties

```
<section>
  <div role="tablist">
    <button role="tab" aria-selected="true">Veg</button>
    <button role="tab" aria-selected="false">Non-Veg</button>
    <button role="tab" aria-selected="false">Vegan</button>
  </div>
  <div role="tabpanel" aria-labelledby="...">...</div>
  <div role="tabpanel" aria-labelledby="...">...</div>
  <div role="tabpanel" aria-labelledby="...">...</div>
</section>
```

#### ROLE

A name to identify by assistive technologies

#### STATE

state is dynamic & needs to update when the user interacts. Screen readers are notified of the changes.

#### PROPERTIES

describes relationships with other elements & they do not change once they're set.

/hellonehha

Let's see how to make tabs accessible for the screen readers? We don't have the tab tag in HTML. Hence, screen readers won't be able to identify what it is? For them it'll be just `div`. To make them, identified by screen

readers we will give them names by using `role`. Now, screen readers will read aloud this as well as this will also tell assistive technologies that it should follow the `tabs` pattern.

ARIA properties support the role of assistive technology. These properties are static and explain the 'nature' or 'relationship' of the elements/tag to the screen readers. ARIA Properties are not expected to be change most of the time. Eg: `aria-label` will speak aloud the `aria-label` even though the text is already provided by assistive technologies. Similarly, `aria-labelledby` will make the relationship between the tabpanel and corresponding tabpanel, one more is `aria-required` will inform the user of assistive technology this input is required.

Next, interaction. Visually it is easy to see what happened when the user interacted with the tabs but for screen readers, it is going to be a challenge as we are using non-native HTML tags. Hence, using `aria-state` will help the screen reader users to understand when the interaction happened and what change on the DOM. Eg: in case of tabs - we selected a tab (`aria-selected='true'`) and corresponding tabpanel is shown (`aria-hidden="false"`).

Using ARIA state and changing the value on interacting (by JavaScript) will make screen readers speak aloud the changes that happened in the DOM. In the above example, it will speak - "Food is selected".

## Summary

If you are developing UI modules for which there are no HTML-semantics or HTML standardized tags then you can use ARIA tags with HTML tags. This works changing the Standard Accessibility DOM. ARIA-roles help in identifying the tag by screen readers as well as ARIA-role means that this tag should behave like this. ARIA-properties & state support the role. ARIA-properties are static and won't be accepted to be change and ARIA-state are dynamic.

## Accessibility Learning Resources

Learning accessibility is important is you are a developer, designer, project manager, or QA. The documentation of WCAG could be overwhelming but one cannot ignore that it is official documentation hence, it goes in depth.

If you are looking for on-point resources to learn accessibility, then here is the list which I refer whenever I need to learn accessibility

## Videos

[A11Ycasts](#)

[edx](#)

## **Blogs and Articles**

[WebAIM](#)

[W3C](#)

[24a11y](#)

[Inclusive Components](#)

[MDN](#)